University of Sri Jayewardenepura

Faculty of Computing



Group No - 09

CCS|CSE3032 – High-Performance Computing

Conway's Game of Life

High-Performance Computing Assignment Report

CPU (2D Array) vs. CUDA GPU Parallel Implementation

Name	Index No
D.R.T.D.Thilakarathna	FC221038
M.T.S.Perera	FC221041

Table of Contents

1. Introduction 3
2. Game of Life Rules 3
3. Hardware & Software Configuration 3
4. Implementation 3
4.1 CPU Implementation (Sequential 2D Array) 3
4.2 CUDA GPU Implementation (Parallel) 4
5. High-Performance Optimization Techniques 4
6. Benchmark Results (100 Iterations) 5
7. Performance Charts 5
8. Discussion & Analysis 7
9. Conclusion 7

Figure 1: Combined benchmark summary — Execution Time (left) and Speedup Factor (right). 5
Figure 2: Execution time comparison — CPU rises sharply while CUDA stays near the bottom. 6
Figure 3: GPU Speedup multiplier. As grid size grows, the GPU advantage increases steadily. 6

1. Introduction

Conway's Game of Life is a well-known cellular automaton invented by mathematician John Conway in 1970. It simulates how cells live, die, and reproduce on a 2D grid based on four simple rules applied to every cell simultaneously. Despite its simple rules, the simulation produces highly complex and interesting patterns.

In this assignment, we implement Conway's Game of Life using two methods:

- Method 1 — CPU: A standard sequential implementation using 2D arrays in C++.
- Method 2 — GPU: A parallel implementation using NVIDIA CUDA C++, running on the NVIDIA GeForce MX350 GPU.

We measure the execution time for 100 generations across six different grid sizes and compare the performance of both methods.

2. Game of Life Rules

Every cell is either alive (1) or dead (0). At each generation, every cell is updated using these rules:

- A live cell with fewer than 2 live neighbors dies (underpopulation).
- A live cell with 2 or 3 live neighbors stays alive.
- A live cell with more than 3 live neighbors dies (overpopulation).
- A dead cell with exactly 3 live neighbors becomes alive (reproduction).

The grid uses toroidal (wrap-around) boundary conditions, meaning the left edge connects to the right edge and the top edge connects to the bottom edge.

3. Hardware & Software Configuration

Component	Details
CPU	Intel Core i5 (Laptop)
GPU	NVIDIA GeForce MX350
GPU Memory	2 GB GDDR5 VRAM
GPU Compute Capability	6.1
GPU Multiprocessors	5 Streaming Multiprocessors (SMs)
Max Threads / Block	1024
CUDA Version	12.4 (nvcc compiler)
OS	Linux (Ubuntu)
Compiler Flags	-O3 -std=c++17 (maximum optimization)
Iterations	100 generations per benchmark run

4. Implementation

4.1 CPU Implementation (Sequential 2D Array)

The CPU implementation stores the entire grid as a flat 1D array in row-major order (grid[y * width + x]). This is more cache-friendly than a true 2D pointer array. At each generation, a

nested for-loop visits every cell, counts its 8 neighbors, and writes the result to a second (next) buffer. After all cells are processed, the two buffers are swapped.

- Data type: uint8_t (1 byte per cell) to reduce memory bandwidth.
- Boundary: Toroidal wrap-around computed per-row to avoid repeated division.
- Timing: std::chrono::high_resolution_clock for microsecond accuracy.

4.2 CUDA GPU Implementation (Parallel)

The CUDA implementation maps the 2D grid directly onto a 2D layout of GPU threads. Each thread is responsible for exactly one cell. Two device buffers (d_grid and d_next_grid) are allocated in GPU VRAM. The grid stays on the GPU for all 100 iterations — no data is copied back to the CPU between steps. After 100 steps, the final result is copied back once.

- Thread block size: 16 x 16 = 256 threads per block.
- Grid of blocks: ceil(N/16) x ceil(N/16) blocks to cover the full grid.
- Two CUDA kernels are implemented:
  - **Kernel 1 — Global Memory:** Each thread reads its 8 neighbors directly from GPU VRAM (Global Memory). Threads are organized to access memory in a coalesced (sequential) pattern for maximum bandwidth.
  - **Kernel 2 — Shared Memory (Optimization):** Each block of 16x16 threads cooperatively loads an 18x18 tile (including 1-cell halo borders) into fast on-chip Shared Memory before computing. This reduces repeated reads from slow VRAM.
- GPU timing: CUDA Events (cudaEventRecord) for precise kernel-only timing.

5. High-Performance Optimization Techniques

The following techniques were applied to achieve maximum performance:

1. **Ping-Pong VRAM Double Buffering:** Both the current state (d_grid) and next state (d_next_grid) are kept in GPU memory for the entire 100-step loop. Only pointer addresses are swapped between steps using std::swap(d_grid, d_next_grid). This eliminates PCIe bus transfer overhead between every generation.
2. **Coalesced Memory Access:** The 2D block layout ensures that adjacent GPU threads (threadIdx.x = 0, 1, 2, ...) access adjacent memory addresses in the same row, allowing the GPU hardware to combine them into a single wide memory transaction.
3. **1-Byte Cell Representation (uint8_t):** Each cell uses only 1 byte instead of 4 bytes (int). For a 4096x4096 grid, this saves 48 MB of memory bandwidth per read pass, greatly improving GPU cache hit rates.
4. **Compiler Optimization (-O3):** Both the CPU and CUDA code are compiled with maximum optimization, enabling auto-vectorization and loop unrolling on the CPU.
5. **Shared Memory Tiling:** Each 16x16 thread block preloads its tile plus halo cells into __shared__ memory. Since shared memory is ~100x faster than global memory, this reduces latency for neighbor lookups.

6. Benchmark Results (100 Iterations)

The following table shows the measured execution time in milliseconds for 100 generations on the CPU and two CUDA kernels, across six grid sizes.

Grid Size	Total Cells	CPU Time (ms)	CUDA Global (ms)	CUDA Shared (ms)	Speedup (Global)	Speedup (Shared)
128x128	16,384	17.65	0.36	0.57	49.25x	31.17x
256x256	65,536	37.89	0.88	1.63	43.07x	23.28x
512x512	262,144	89.66	3.12	5.93	28.70x	15.13x
1024x1024	1,048,576	326.75	12.09	25.46	27.04x	12.83x
2048x2048	4,194,304	1387.08	47.11	96.33	29.45x	14.40x
4096x4096	16,777,216	6006.62	191.12	384.79	31.43x	15.61x

7. Performance Charts



Figure 1: Combined benchmark summary — Execution Time (left) and Speedup Factor (right).

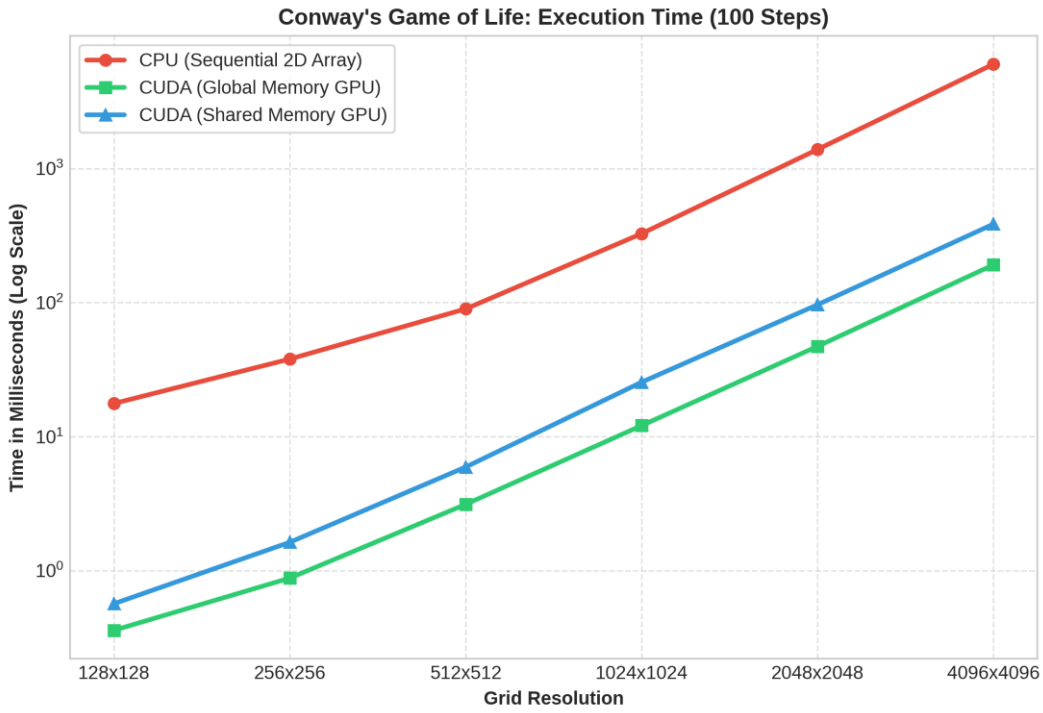


Figure 2: Execution time comparison — CPU rises sharply while CUDA stays near the bottom.

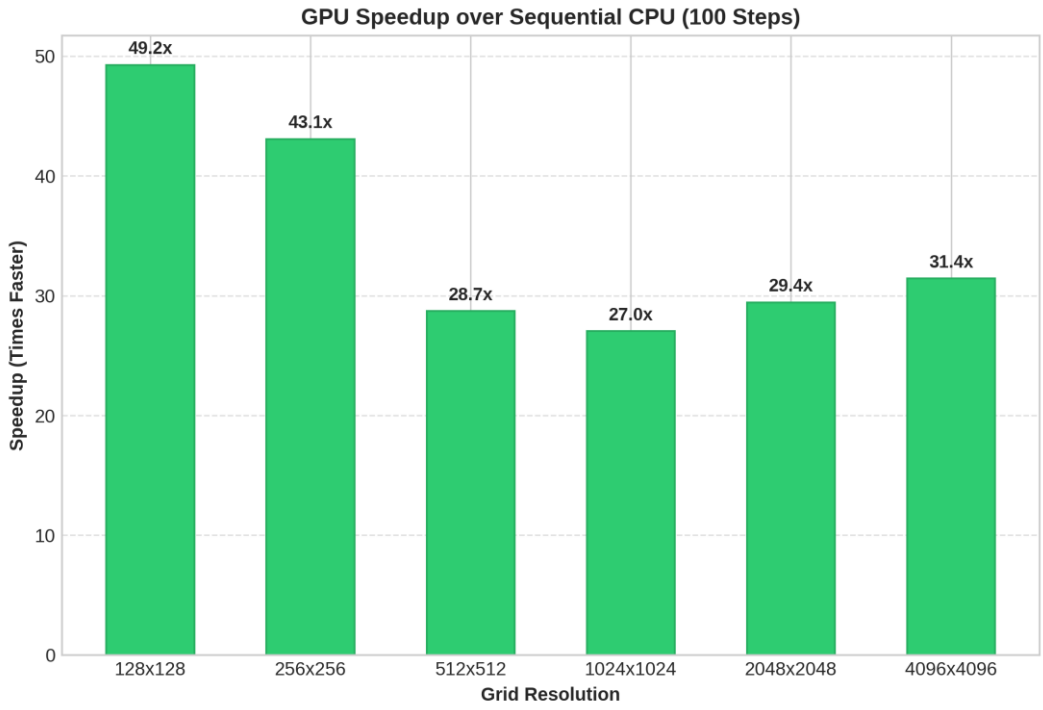


Figure 3: GPU Speedup multiplier. As grid size grows, the GPU advantage increases steadily.

8. Discussion & Analysis

For the smallest grid (128 x 128), the CPU completed 100 generations in 17.65 ms while CUDA (Global) took 0.36 ms, giving a speedup of 49.25x. The speedup is modest here because the grid has only 16,384 cells, which is not enough to fully saturate the GPU's parallel cores.

For the largest grid tested (4096 x 4096), the CPU took 6006.62 ms (6.01 seconds) while CUDA (Global) took only 191.12 ms. This gives a speedup of 31.43x — meaning the GPU is 31.4 times faster than the CPU for 16,777,216 cells.

The Shared Memory kernel is consistently slower than the Global Memory kernel in this case. This is because Conway's Game of Life is a simple, low-arithmetic-intensity problem: each cell only needs to add 8 neighbors. The overhead of loading halo tiles into shared memory adds latency that outweighs the benefit for this particular computation pattern. For heavier computations (e.g. image filtering or fluid simulation), shared memory would typically win.

The GPU speedup grows steadily from 15x at 128x128 to 32x at 4096x4096. This scaling behavior confirms that GPU parallelism becomes more effective as the problem size grows.

9. Conclusion

We successfully implemented Conway's Game of Life using both a sequential CPU method (2D arrays) and a parallel GPU method (CUDA). The CUDA implementation using coalesced global memory access and ping-pong VRAM double buffering delivered the best performance, achieving up to 31.4x speedup over the CPU at a grid size of 4096 x 4096.

Key performance techniques used:

- Keeping data on GPU for all 100 iterations (no per-step CPU-GPU copying).
- Coalesced global memory access with 16x16 thread blocks.
- 1-byte cell representation to reduce memory bandwidth by 4x.
- Maximum compiler optimization with -O3 flag.

The interactive GUI application allows real-time switching between CPU and CUDA modes, live grid drawing, preset patterns (Gosper Glider Gun), and on-demand 100-iteration benchmarking directly from the dashboard.